

PEP 679 – New assert statement syntax with parentheses

Author: Pablo Galindo Salgado <pablogsal at python.org>, Stan Ulbrych <stan at python.org>

Discussions-To: Discourse thread

Status: Rejected

Type: Standards Track

Created: 07-Jan-2022

Python-Version: 3.15

Post-History: 08-Sep-2025, 10-Jan-2022

Resolution: 24-Oct-2025

Abstract

This PEP proposes allowing parentheses in the two-argument form of `assert`. The interpreter will reinterpret `assert (expr, msg)` as `assert expr, msg`, eliminating the common pitfall where such code was previously treated as asserting a two-element tuple, which is always truthy.

Motivation

It is a common user mistake when using the form of the `assert` statement that includes the error message to surround it with parentheses [1] [2]. This is because many beginners assume `assert` is a function. The prominent `unittest` methods, particularly `assertTrue()`, also require parentheses around the assertion and message.

Unfortunately, this mistake passes undetected as the `assert` will always pass [6], because it is interpreted as an `assert` statement where the expression is a two-tuple, which always has truth-y value. The mistake also often occurs when extending the test or description beyond a single line, as parentheses are a natural way to do that.

This is so common that a `SyntaxWarning` is emitted by the compiler since 3.10 and several code linters [3] [4].

Additionally, some other statements in the language allow parenthesized forms in one way or another, for example, `import` statements (`from x import (a,b,c)`) or `del` statements (`del (a,b,c)`).

Allowing parentheses not only will remove the pitfall but also will allow users and auto-formatters to format long `assert` statements over multiple lines in what the authors of this document believe will be a more natural way. Although it is possible to currently format long `assert` statements over multiple lines with backslashes (as is recommended by PEP 8) or parentheses and a comma:

```
assert (  
    very very long  
    test  
) , (  
    "very very long "  
    "error message"  
)
```

the authors of this document believe the proposed parenthesized form is more clear and intuitive, along with being more consistent with the formatting of other grammar constructs:

```
assert (  
    very very long  
    test,  
  
    "very very long "  
    "message"  
)
```

Rationale

Due to backwards compatibility concerns (see section below), to inform users of the new change of how what was previously a two element tuple is parsed, a `SyntaxWarning` with a message like "new assertion syntax, will assert first element of tuple" will be raised till Python 3.17. For example, when using the new syntax:

```
>>> assert ('Petr' == 'Pablo', "That doesn't look right!")
<python-input-0>:0: SyntaxWarning: new assertion syntax, will assert f1
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    assert ('Petr' == 'Pablo', "That doesn't look right!")
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: That doesn't look right!
```



Note that improving syntax warnings in general is out of the scope of this PEP.

Specification

The formal grammar of the `assert` statement will change to [8]:

```
| 'assert' '(' expression ',' expression [' ',''] ')' &(NEWLINE | ';' )
| 'assert' a=expression [' ',''] expression ]
```

Where the first line is the new form of the `assert` statement that allows parentheses and will raise a `SyntaxWarning` till 3.17. The lookahead is needed to prevent the parser from eagerly capturing the tuple as the full statement, so statements like `assert (a, b) <= c, "something"` are still parsed correctly.

Implementation Notes

This change can be implemented in the parser or in the compiler. The specification that a `SyntaxWarning` be raised informing users of the new syntax complicates the implementation, as warnings should be raised during compilation.

The authors believe that an ideal implementation would be in the parser [8], resulting in `assert (x,y)` having the same AST as `assert x,y`. This necessitates a two-step implementation plan, with a necessary temporary compromise.

Implementing in the parser

It is not possible to have a pure parser implementation with the warning specification. (Note that, without the warning specification the pure parser implementation is a small grammar change [5]). To raise the warning, the compiler must be aware of the new syntax, this means that an optional flag would be necessary as otherwise the information is lost during parsing. As such, the AST of an `assert` with parentheses would look like so, with a `paren_syntax=1` flag:

```
>>> print(ast.dump(ast.parse('assert(True, "Error message)'), indent=4)
Module(
  body=[
    Assert(
      test=Constant(value=True),
      msg=Constant(value='Error message'),
      paren_syntax=1)])
```



Implementing in the compiler

The new syntax can be implemented in the compiler by special casing tuples of length two. This however, will have the side-effect of not modifying the AST whatsoever during the transition period while the `SyntaxWarning` is being emitted.

Once the `SyntaxWarning` is removed, the implementation can be moved to the parser level, where the parenthesized form would be parsed directly into the same AST structure as `assert expression, message`. This approach is more backwards-compatible, as the many tools that deal with ASTs will have more time to adapt.

Backwards Compatibility

The change is not technically backwards compatible. Whether implemented initially in the parser or the compiler, `assert (x,y)`, which is currently interpreted as an assert statement with a 2-tuple as the subject and is always truth-y, will be interpreted as `assert x,y`.

On the other hand, assert statements of this kind always pass, so they are effectively not doing anything in user code. The authors of this document think that this backwards incompatibility nature is beneficial, as it will highlight these cases in user code while before they will have passed unnoticed. This case has already raised a `SyntaxWarning` since Python 3.10, so there has been a deprecation period of over 5 years. The continued raising of a `SyntaxWarning` should mitigate surprises.

The change will also result in changes to the AST of `assert (x,y)`, which currently is:

```

Module(
    body=[
        Assert(
            test=Tuple(
                elts=[
                    Name(id='x', ctx=Load()),
                    Name(id='y', ctx=Load())],
                ctx=Load()))],
    type_ignores=[])

```

the final implementation, in Python 3.18, will result in the following AST:

```

Module(
    body=[
        Assert(
            test=Name(id='x', ctx=Load()),
            msg=Name(id='y', ctx=Load()))],
    type_ignores=[])

```

The problem with this is that the AST of the first form will technically be “incorrect” as we already have a specialized form for the AST of an assert statement with a test and a message (the second one). Implementing initially in the compiler will delay this change, alleviating backwards compatibility concerns, as tools will have more time to adjust.

How to Teach This

The new form of the `assert` statement will be documented as part of the language standard.

When teaching the form with error message of the `assert` statement to users, now it can be noted that adding parentheses also work as expected, which allows to break the statement over multiple lines.

Reference Implementation

A reference implementation in the parser can be found in this [branch](#) and reference implementation in the compiler can be found in this [branch](#).

Rejected Ideas

Adding a syntax with a keyword

Everywhere else in Python syntax, the comma separates variable-length “lists” of homogeneous elements, like the the items of a `tuple` or `list`, parameters/arguments of functions, or import targets. After Python 3.0 introduced `except...as`, the `assert` statement remains as the only exception to this convention.

It's possible that user confusion stems, at least partly, from an expectation that comma-separated items are equivalent. Enclosing an `assert` statement's expression and message in parentheses would visually bind them together even further. Making `assert` look more similar to a function call encourages a wrong mentality.

As a possible solution, it was proposed [7] to replace the comma with a keyword, and the form would allow parentheses, for example:

```
assert condition else "message"  
assert (condition else "message")
```

The comma could then be slowly and carefully deprecated, starting with the case where they appear in parentheses, which already raises a `SyntaxWarning`.

The authors of this PEP believe that adding a completely new syntax will, first and foremost, not solve the common beginner pitfall that this PEP aims to patch, and will not improve the formatting of `assert` statements across multiple lines, which the authors believe the proposed syntax improves.

Security Implications

There are no security implications for this change.

Acknowledgements

This change was originally discussed and proposed in [python/cpython#90325](https://github.com/python/cpython/issues/90325).

Many thanks to Petr Viktorin for his help during the drafting process of this PEP.

Footnotes

[1]

StackOverflow: “‘assert’ statement with or without parentheses”

[2]

/r/python: “Rant: use that second expression in assert! “

[3]

flake8: Rule F631

[4]

pylint: assert-on-tuple (W0199)

[5]

For the previous parser implementation, see [python/cpython#30247](#)

[6]

During the updating of this PEP, an exception (`assert (*(t := ()),)`) was found, contradicting the warning.

[7]

[DPO] Pre-PEP: Assert-with: Dedicated syntax for assertion messages

[8] (1, 2)

An edge case arises with constructs like:

```
>>> x = (0,)
>>> assert (*x, "edge cases aren't fun:-(")
```

This form is currently parsed as a single tuple expression, not as a condition/message pair, and will need explicit handling in the compiler.

Copyright

This document is placed in the public domain or under the CC0-1.0-Universal license, whichever is more permissive.

Source: <https://github.com/python/peps/blob/main/peps/pep-0679.rst>

Last modified: 2026-03-30 01:04:04 GMT